

Proyecto final.

Alisson Daniela Molina Ibarra

Dylan Humberto Trejo Mejía

Facultad de Ingeniería, Institución Universitaria Pascual Bravo

Herramientas de Programación II

Docente Alexandra Guerrero Bocanegra

26 de mayo de 2026

1. Proyecto

Sistema de control de insumos y abastecimiento.

2. Nombre del proyecto

Supply Manager

3. Resumen ejecutivo

Supply Manager es un sistema de escritorio desarrollado en C# con Windows Forms y .NET Framework 4.7.2, cuyo propósito es centralizar y automatizar el control de insumos y abastecimiento en organizaciones que actualmente gestionan sus inventarios de forma manual o con hojas de cálculo.

El sistema implementa una arquitectura de tres capas (presentación, lógica de negocio y acceso a datos), aplica los cinco principios SOLID y tres patrones de diseño GoF (Singleton, Factory Method y Repository). Toda la interacción con la base de datos se realiza mediante procedimientos almacenados en SQL Server, garantizando integridad transaccional y reglas de negocio centralizadas.

4. Planteamiento del problema

La falta de trazabilidad en el consumo de insumos genera sobre costos, desabastecimiento crítico y falta de control sobre quién retira qué material y/o productos, pérdidas de inventario, duplicidad de información, dificultad para generar reportes, etc. Adicionalmente, muchas empresas y pequeños negocios aún realizan control de los insumos por medio de herramientas como Excel e incluso de manera manual, lo que puede generar errores, una

notable disminución de la eficiencia y desaprovechamiento del tiempo operativo. Es por ello que resulta fundamental sistematizar este tipo de procesos para optimizar los recursos y el tiempo.

5. Justificación

Supply Manager responde directamente a los problemas identificados mediante un sistema accesible (aplicación de escritorio sin dependencias externas) que:

- Centraliza toda la información de insumos, movimientos y proveedores en una base de datos relacional.
- Audita cada movimiento con fecha, usuario, área y observación.
- Alerta sobre stock crítico en tiempo real.
- Proyecta la fecha estimada de agotamiento y la cantidad sugerida de reorden, usando el consumo promedio de los últimos 30 días.
- Restringe el acceso mediante un sistema de roles (Administrador, Almacenista, Supervisor), evitando modificaciones no autorizadas.
- Aplica prácticas de ingeniería de software (SOLID, GoF, arquitectura por capas) que facilitan el mantenimiento y la extensión futura del sistema.

El proyecto también sirve como evidencia académica de diseño orientado a objetos aplicado a un caso de uso real.

6. Objetivos del sistema

- **Objetivo general:**

Desarrollar un MVP de escritorio en C# Windows Forms para gestionar el control de insumos y abastecimiento empresarial, permitiendo administrar inventario, entradas,

salidas y niveles mínimos, mediante una arquitectura por capas, garantizando la integridad de los datos y facilitando la toma de decisiones con proyecciones de reabastecimiento.

- **Objetivos específicos:**

- ✓ Gestionar inventario de insumos.
- ✓ Controlar entradas y salidas.
- ✓ Administrar proveedores.
- ✓ Validar disponibilidad de stock.
- ✓ Generar reportes de consumo y stock crítico.
- ✓ Implementar autenticación por roles.
- ✓ Aplicar arquitectura en capas y principios SOLID.
- ✓ Implementar una proyección simple de reabastecimiento.

7. Requisitos funcionales

➤ RF1: Login

El sistema debe permitir autenticación de usuarios con roles (administrador, almacenista, supervisor) mediante usuario y contraseña.

➤ RF2: CRUD de insumos

Debe permitir crear, consultar, editar y eliminar.

➤ RF3: CRUD de proveedores

➤ RF4: CRUD de áreas

➤ RF5: CRUD de movimientos

Entradas y salidas de inventario.

➤ **RF6: Control de Stock**

Validar existencias disponibles.

➤ **RF7: Niveles Mínimos**

Generar alertas cuando el stock sea menor al nivel mínimo.

➤ **RF8: Filtros**

El sistema debe filtrar por insumo, proveedor, área y rango de fechas.

➤ **RF9: Reportes**

Debe generar reportes de consumo, rotación y stock crítico.

➤ **RF10: Función diferencial**

Proyección de reabastecimiento, en el cual debe calcular sugerencia de compra basada en consumo promedio.

8. Requisitos no funcionales

➤ **RF1:** sistema desarrollado en C#, Windows Forms y SQL Server.

➤ **RF2:** arquitectura en capas.

➤ **RF3:** uso de principios SOLID.

➤ **RF4:** implementación de patrones GoF.

➤ **RF5:** persistencia segura mediante procedimientos almacenados.

➤ **RF6:** implementación de patrones GoF.

➤ **RF7:** validaciones robustas.

➤ **RF8:** interfaz intuitiva.

➤ **RF9:** seguridad por roles.

9. Reglas de negocio

- **RN1:** el sistema no debe permitir registrar una salida de insumos cuando la cantidad solicitada supera el stock disponible.
- **RN2:** todo insumo debe tener definido un nivel mínimo de existencia mayor a cero.
- **RN3:** cuando el stock actual de un insumo sea menor o igual a su nivel mínimo, el sistema debe mostrar una alerta visual de abastecimiento.
- **RN4:** el sistema no permitirá registrar movimientos con fechas futuras.
- **RN5:** no pueden existir proveedores registrados con el mismo nombre e identificación.
- **RN6:** toda operación realizada sobre inventario debe quedar registrada.
- **RN7:** el sistema debe calcular una sugerencia de reabastecimiento basada en el promedio de consumo histórico del insumo.

10. Arquitectura

El sistema de control de insumos y abastecimiento será desarrollado bajo una arquitectura en capas, con el objetivo de garantizar una adecuada separación de responsabilidades, facilitar el mantenimiento del software, mejorar la escalabilidad y promover buenas prácticas de ingeniería de software.

La arquitectura propuesta permite desacoplar la interfaz gráfica, la lógica de negocio y el acceso a datos, favoreciendo la reutilización de componentes y la organización estructurada del proyecto.

El sistema será implementado utilizando C# Windows Forms como tecnología de interfaz de usuario y SQL Server como sistema gestor de base de datos relacional.

La solución se compone de las siguientes capas: presentación (UI), lógica de Negocio (BLL), acceso a Datos (DAL) y Base de Datos (SQL Server).

- Capa de Presentación (UI): la capa de presentación corresponde a la interfaz gráfica desarrollada en Windows Forms. Su responsabilidad principal es interactuar con el usuario, capturar información y mostrar resultados del sistema.

En esta capa se implementarán formularios del sistema, validaciones básicas de entrada, navegación entre módulos y visualización de reportes y alertas.

- Capa de Lógica de Negocio (BLL): la capa de lógica de negocio contiene las reglas, políticas y procesos principales del sistema. Es la encargada de validar la información recibida desde la interfaz y coordinar las operaciones necesarias antes de acceder a los datos.

Entre sus responsabilidades se encuentran: validación de reglas de negocio, control de stock disponible, validación de cantidades y fechas, generación de alertas de stock crítico y cálculo de proyección de reabastecimiento.

- Capa de Acceso a Datos (DAL): la capa de acceso a datos tiene la responsabilidad de gestionar la comunicación con la base de datos SQL Server. Su función es ejecutar consultas, procedimientos almacenados y operaciones CRUD necesarias para la persistencia de la información. Esta capa se implementará utilizando ADO.NET y procedimientos almacenados para garantizar mayor seguridad, mantenibilidad y control sobre las operaciones de base de datos. Algunas de sus responsabilidades son la conexión a la base de datos, ejecución de procedimientos almacenados, consultas y recuperación de información, inserción, actualización y eliminación de registros.

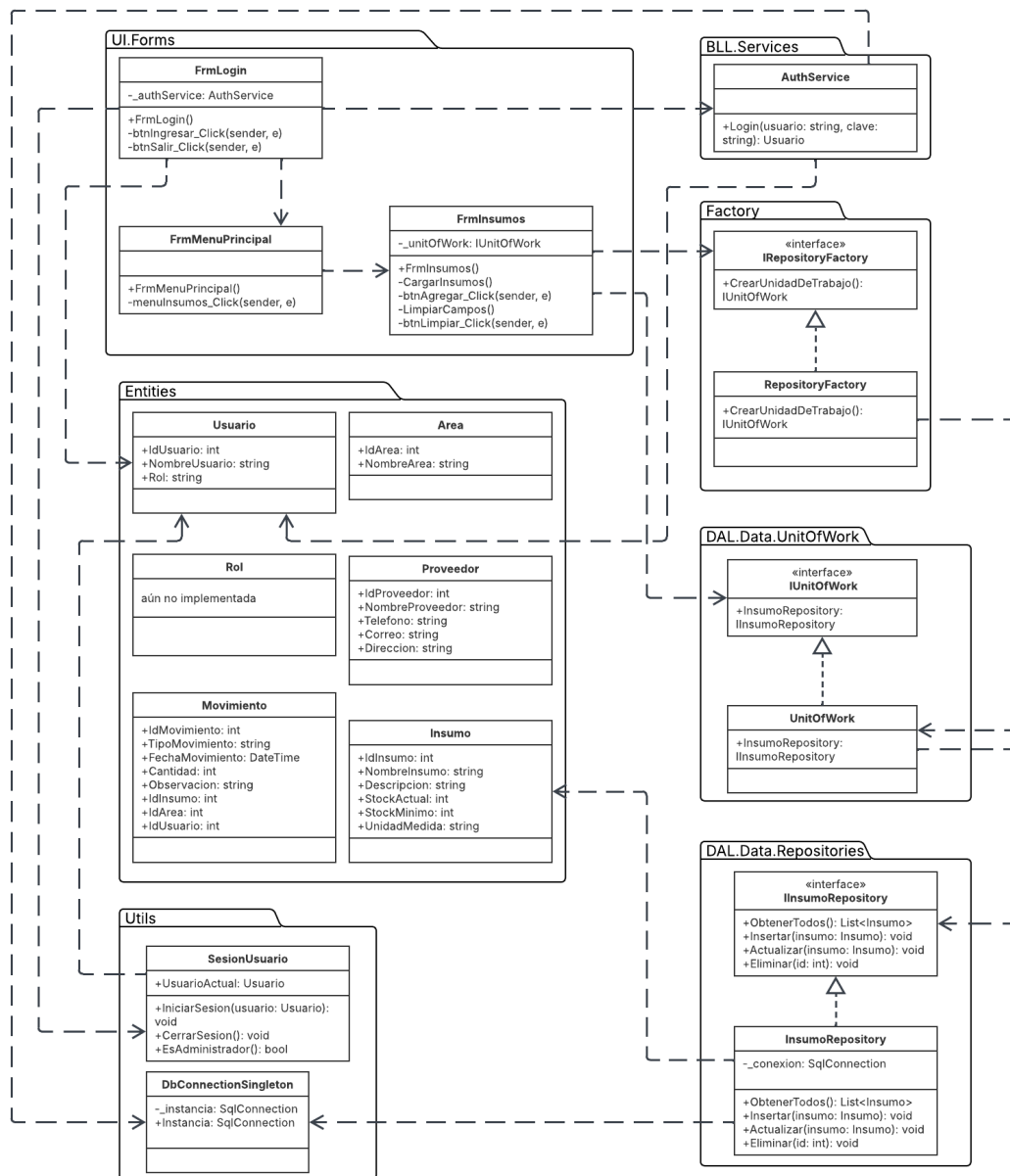
- Base de Datos: la base de datos será implementada en SQL Server bajo un modelo relacional normalizado hasta tercera forma normal (3FN), con el propósito de reducir redundancia y mantener integridad de la información. La base de datos almacenará información sobre usuarios, roles, insumos, proveedores, movimientos de inventario y niveles mínimos de stock.

11. Modelo orientado a objetos

El sistema aplica los siguientes principios del paradigma orientado a objetos:

- Abstracción: cada entidad del dominio (Insumo, Movimiento, Proveedor, etc.) está representada como una clase C# con propiedades que capturan solo los atributos relevantes para el sistema.
- Encapsulamiento: las clases de repositorio ocultan los detalles de ADO.NET detrás de interfaces (`IInsumoRepository``, `IMovimientoRepository``, etc.). Los formularios no conocen la implementación concreta.
- Herencia: todos los formularios heredan de `System.Windows.Forms.Form``. Los servicios podrían heredar de una clase base `ServiceBase`` en una versión futura.
- Polimorfismo: las referencias a `IUnitOfWork`` en los servicios permiten sustituir la implementación concreta sin cambiar el código del servicio (base para pruebas unitarias).
- Interfaces como contratos: `IRepositoryFactory``, `IUnitOfWork``, `IInsumoRepository``, etc., definen contratos que desacoplan las capas.

12. Diagrama de clases



13. Principios SOLID Aplicados: durante el desarrollo del sistema se aplicarán principios

SOLID con el fin de mejorar la calidad y mantenibilidad del software. Los usados en el proyecto son:

- ✓ Single Responsibility Principle (SRP): cada clase tendrá una única responsabilidad.

- ✓ Open/Closed Principle (OCP): los módulos estarán preparados para extender funcionalidades sin modificar código existente.
- ✓ Dependency Inversion Principle (DIP): se utilizarán interfaces para desacoplar servicios y repositorios.

14. Patrones de Diseño Utilizados: el sistema implementará patrones de diseño GoF para mejorar la organización y reutilización del código.

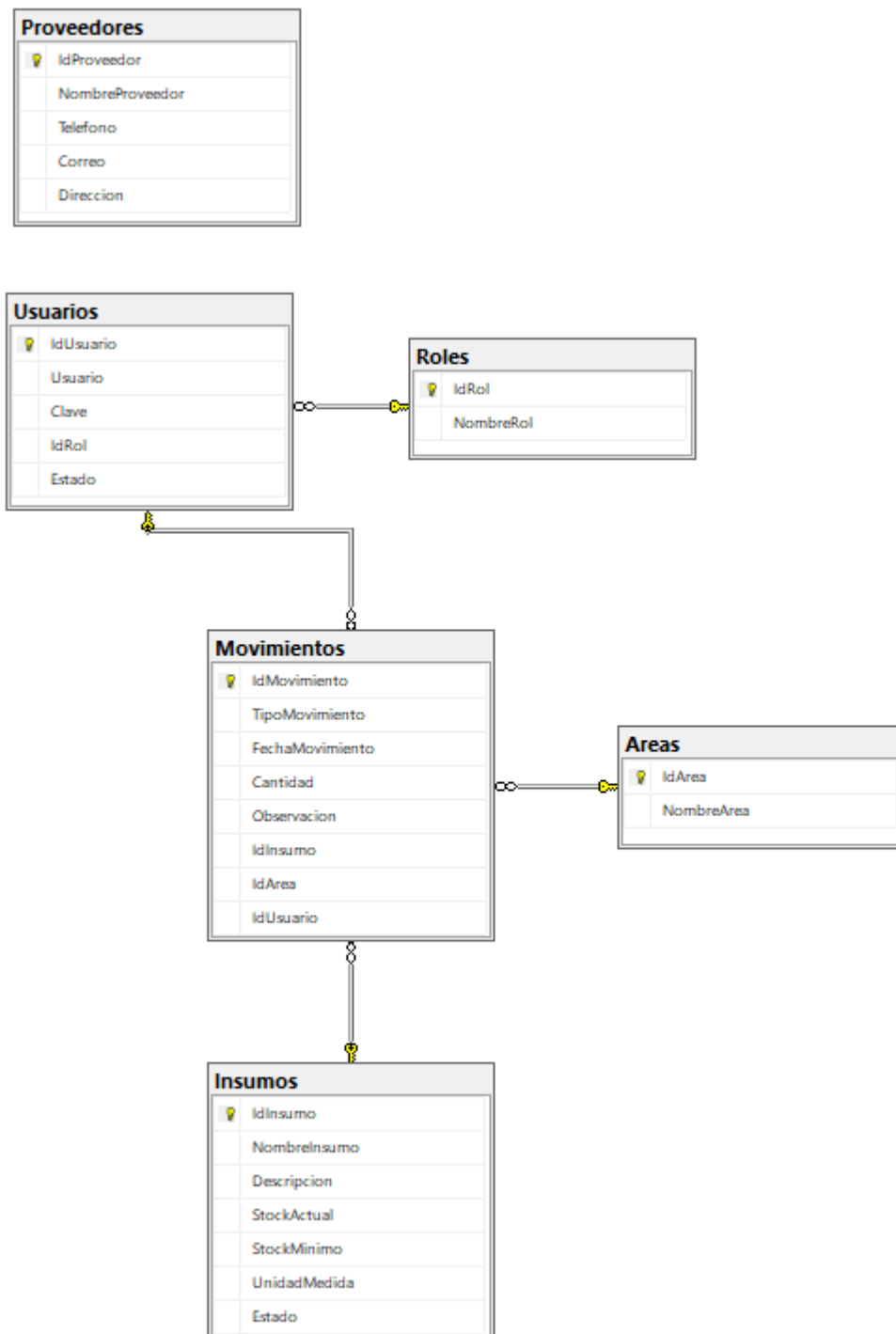
- ✓ Repository Pattern: utilizado para abstraer el acceso a datos, centralizar las operaciones CRUD y desacoplar la capa de presentación de ADO.NET. Cada entidad tiene su interfaz (IInsumoRepository, IMovimientoRepository, etc.) y su implementación concreta.
- ✓ Singleton Pattern: aplicado en DbConnectionSingleton para garantizar una única instancia de SqlConnection en toda la aplicación, controlando el ciclo de vida de la conexión a SQL Server.
- ✓ Factory Method Pattern: implementado mediante IRepositoryFactory y RepositoryFactory, que encapsulan la creación de instancias de UnitOfWork, desacoplando los servicios de la construcción concreta de sus dependencias.
- ✓ Unit of Work Pattern: implementado en IUnitOfWork / UnitOfWork para agrupar todos los repositorios bajo una única dependencia, garantizando coherencia en el uso de la misma conexión y simplificando la inyección de dependencias en los servicios.

Flujo General de Funcionamiento:

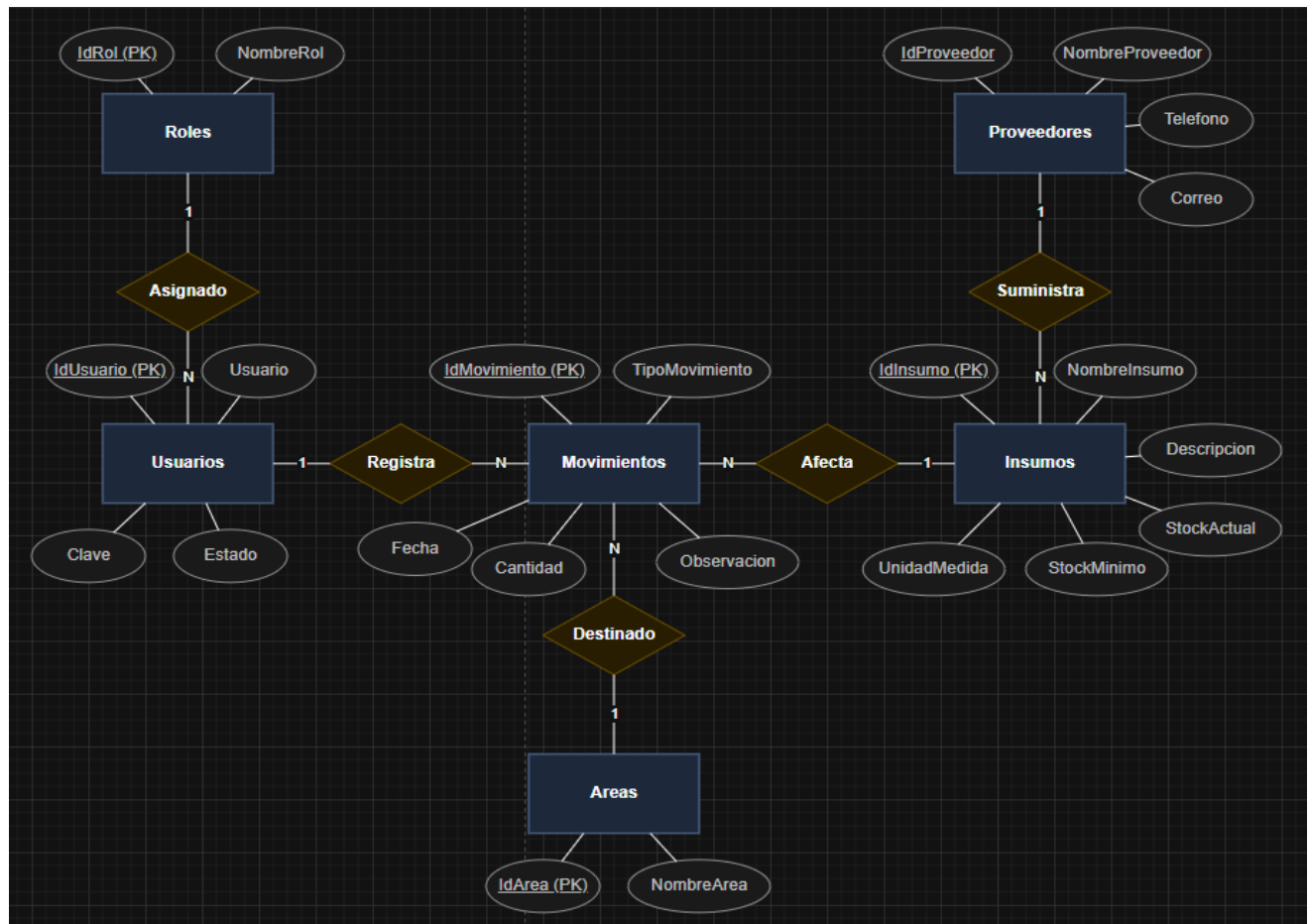
- ✓ El usuario interactúa con la interfaz gráfica.
- ✓ La capa de presentación envía la solicitud a la lógica de negocio.
- ✓ La lógica de negocio valida reglas y restricciones.

- ✓ La capa de acceso a datos ejecuta operaciones sobre SQL Server.
- ✓ Los resultados son retornados a la interfaz para su visualización.

15. Diseño de base de datos



16. Diagrama modelo entidad relación



17. Normalización de la base de datos

- Primera Forma Normal (1FN)

Condición: cada celda contiene un valor atómico; no hay grupos repetitivos.

- ✓ Todas las columnas almacenan un único valor atómico (no hay listas ni arrays en ninguna columna).
- ✓ Cada tabla tiene clave primaria definida (IDENTITY).
- ✓ No existen columnas multi-valor.

Ejemplo: la tabla `Movimientos` no almacena múltiples insumos en un campo; cada movimiento es una fila individual con un único IdInsumo.

- Segunda Forma Normal (2FN)

Condición: cumple 1FN y todos los atributos no clave dependen funcionalmente de toda la clave primaria (aplica cuando la PK es compuesta).

- ✓ Todas las tablas tienen PK simple (INT IDENTITY), por lo que 2FN se cumple trivialmente: no existe dependencia parcial posible.

- Tercera Forma Normal (3FN)

Condición: cumple 2FN y no existen dependencias transitivas (atributo no clave que depende de otro atributo no clave).

- ✓ Usuarios: `NombreRol` no se almacena en `Usuarios`; se almacena `IdRol` (FK a `Roles`). Se elimina la dependencia transitiva $\text{IdUsuario} \rightarrow \text{IdRol} \rightarrow \text{NombreRol}$.
- ✓ Movimientos: los nombres de insumo, área y usuario no se repiten en `Movimientos`; se almacenan como FK. Las uniones se hacen en las consultas (JOIN) o en los SPs.
- ✓ Insumos: no existe redundancia. `StockActual` y `StockMinimo` dependen directamente del `IdInsumo`.

18. Procedimientos almacenados

- **sp_Login:** valida credenciales y retorna datos del usuario y su rol. Solo usuarios activos.
- **sp_InsertarInsumo:** inserta un insumo nuevo. Valida que no exista otro activo con el mismo nombre.

- **sp_ActualizarInsumo:** actualiza nombre, descripción, stock mínimo y unidad. No modifica StockActual directamente.
- **sp_DesactivarInsumo:** desactivación lógica (Estado = 0). El insumo queda como historial.
- **sp_ObtenerInsumos:** consulta con filtros opcionales: nombre (LIKE), unidad de medida y stock crítico.
- **sp_RegistrarMovimiento (principal):** valida stock para salidas, actualiza StockActual e inserta el movimiento atómicamente.
- **sp_ObtenerMovimientos:** consulta con 5 filtros opcionales: insumo, área, tipo, fecha desde, fecha hasta.
- **sp_InsertarProveedor:** inserta un proveedor nuevo.
- **sp_ActualizarProveedor:** actualiza todos los campos del proveedor.
- **sp_EliminarProveedor:** eliminación física (los proveedores no tienen FK en esta versión).
- **sp_ObtenerProveedores:** consulta con filtro opcional por nombre (LIKE).
- **sp_InsertarArea:** inserta un área. Valida nombre único.
- **sp_ActualizarArea:** actualiza nombre. Valida nombre único.
- **sp_EliminarArea:** elimina un área solo si no tiene movimientos asociados; de lo contrario lanza error.
- **sp_ObtenerAreas:** retorna todas las áreas ordenadas alfabéticamente.
- **sp_InsertarUsuario:** inserta usuario con rol. Valida nombre único.
- **sp_ActualizarUsuario:** actualiza nombre de usuario y rol. Valida nombre único.
- **sp_DesactivarUsuario:** desactivación lógica del usuario (Estado = 0).

- **sp_ObtenerUsuarios:** retorna todos los usuarios con su rol (JOIN Roles).
- **sp_ObtenerRoles:** retorna los roles disponibles para poblar combobox.
- **sp_ReporteStockCritico:** insumos con $\text{StockActual} \leq \text{StockMinimo}$, ordenados por déficit descendente.
- **sp_ReporteConsumo:** agrega entradas y salidas por insumo y área en un rango de fechas.
- **sp_ProyeccionReabastecimiento (Función diferencial discreta):** calcula consumo promedio diario (últimos 30 días) y proyecta días hasta nivel mínimo y cantidad sugerida de reorden.

19. Manejo de errores y validaciones

- Cadena de manejo de errores

SQL Server (THROW 5XXXX)

→ Repository (catch `SqlException.Number >= 50000` → throw `ApplicationException`)

→ Service (validaciones de negocio → throw `ArgumentException` /

`UnauthorizedException`)

→ Form (catch todas → `MessageBox.Show(ex.Message)`)

Tipos de errores por capa

- SQL Server (errores de usuario, código ≥ 50000):

50001: ya existe un insumo activo con ese nombre.

50002: ya existe otro insumo con ese nombre.

50010: stock insuficiente para realizar la salida.

50020: ya existe un área con ese nombre.

50021: ya existe otra área con ese nombre.

50022: no se puede eliminar el área porque tiene movimientos asociados.

50030: ya existe un usuario con ese nombre.

50031: ya existe otro usuario con ese nombre.

- Capa BLL (validaciones antes de llegar a BD):

- Campo vacío o nulo (ArgumentException)
- Valores negativos de stock (ArgumentException)
- Formato de correo inválido (ArgumentException)
- Cantidad ≤ 0 en movimiento (ArgumentException)
- Fecha futura en movimiento (ArgumentException)
- Operación no autorizada para el rol (UnauthorizedAccessException)
- Intento de desactivar el propio usuario (InvalidOperationException)

- Propagación hacia el formulario:

Todos los formularios usan el mismo patrón:

```
try
```

```
{
```

```
    _service.Agregar(entidad);
```

```
    MessageBox.Show("Registro exitoso.", "Éxito", ..., MessageBoxIcon.Information);
```

```
    CargarDatos();
```

```
}
```

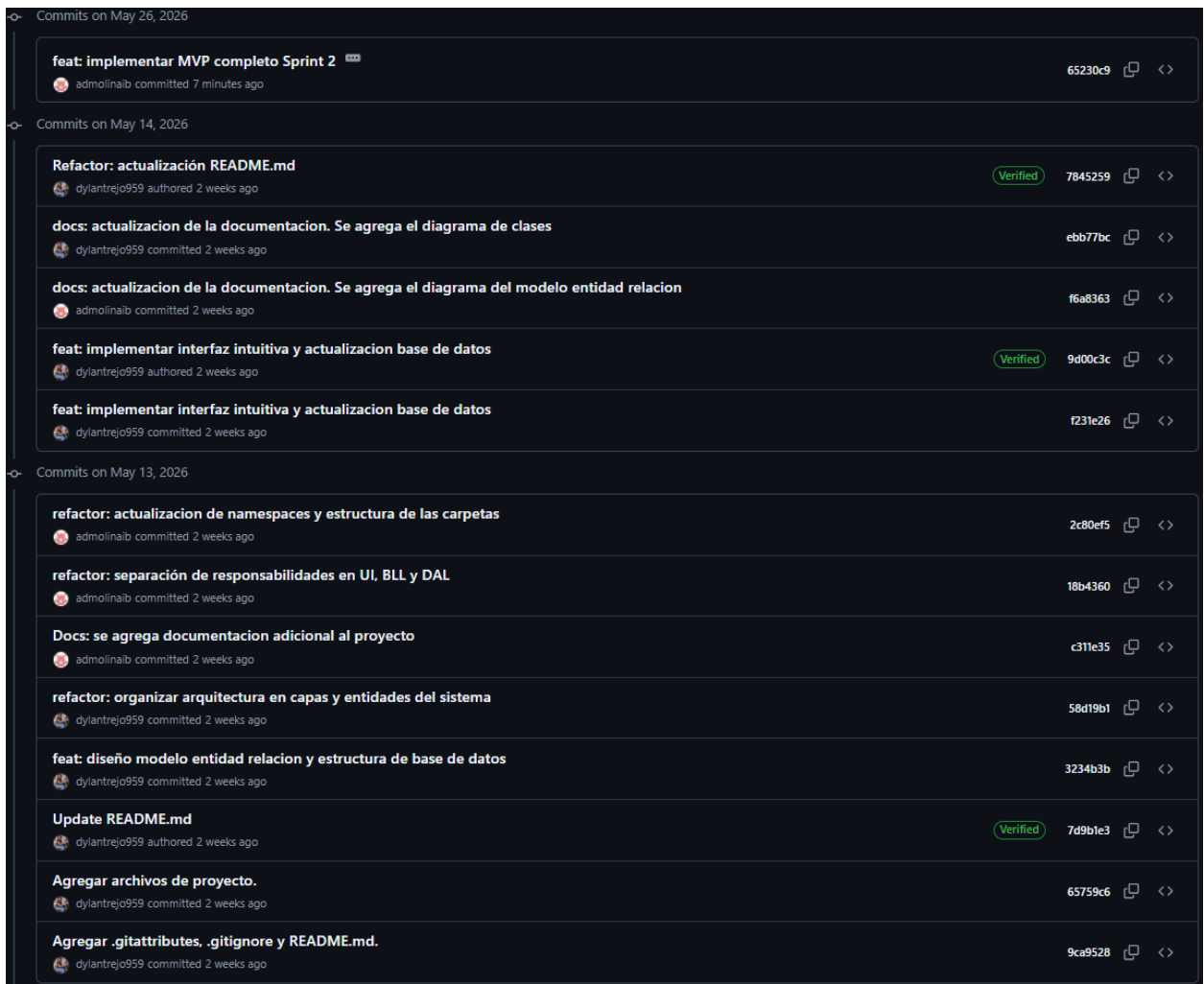

catch (Exception ex)

{

 MessageBox.Show(ex.Message, "Error", ..., MessageBoxIcon.Warning);

}

20. Evidencia de Github



21. Casos de prueba

- CP01 — Autenticación exitosa (Administrador)
 - Precondición: usuario `admin` existe en BD con Estado = 1, rol = Administrador
 - Entrada: usuario: `admin`, Contraseña: `123`
 - Acción: clic en "Ingresar"
 - Resultado esperado: se abre FrmMenuPrincipal con todos los menús visibles, incluido "Usuarios"

- CP02 — Autenticación fallida (credenciales incorrectas)
 - Entrada: usuario: `admin`, Contraseña: `999`
 - Acción: clic en "Ingresar"
 - Resultado esperado: mensaje: "Usuario o contraseña incorrectos". No se abre el menú.

- CP03 — Autenticación con campos vacíos
 - Entrada: usuario: vacío, Contraseña: vacío
 - Resultado esperado: mensaje: "Debe completar todos los campos"

- CP04 — Supervisor no ve botones de modificación en Insumos
 - Precondición: sesión iniciada con usuario `supervisor`
 - Acción: abrir módulo Insumos

- Resultado esperado: botones "Agregar", "Actualizar", "Desactivar" deshabilitados (grisados)

- CP05 — Registro de movimiento de salida con stock suficiente
 - Precondición: insumo "Guantes" con StockActual = 100
 - Entrada: tipo: Salida, Insumo: Guantes, Área: Laboratorio, Cantidad: 10
 - Resultado esperado: movimiento registrado; StockActual de "Guantes" = 90

- CP06 — Registro de salida con stock insuficiente
 - Precondición: insumo "Mascarillas" con StockActual = 5
 - Entrada: tipo: Salida, Insumo: Mascarillas, Cantidad: 20
 - Resultado esperado: mensaje de error: "Stock insuficiente para realizar la salida."
StockActual sin cambios.

- CP07 — Agregar insumo con nombre duplicado
 - Precondición: insumo "Guantes" ya existe con Estado = 1
 - Entrada: nombreInsumo: "Guantes", unidad: "Caja"
 - Resultado esperado: mensaje: "Ya existe un insumo activo con ese nombre."

- CP08 — Reporte de stock crítico
 - Precondición: insumo "Mascarillas" con StockActual = 5, StockMinimo = 15
 - Acción: módulo Reportes → tab "Stock Crítico" → clic "Generar reporte"
 - Resultado esperado: grid muestra "Mascarillas" con Déficit = 10

- CP09 — Eliminar área con movimientos asociados
 - Precondición: área "Laboratorio" tiene movimientos registrados
 - Acción: módulo Áreas → seleccionar "Laboratorio" → clic "Eliminar"
 - Resultado esperado: mensaje: "No se puede eliminar el área porque tiene movimientos asociados."

- CP10 — Reporte de proyección sin historial de consumo
 - Precondición: insumo nuevo sin movimientos en los últimos 30 días
 - Acción: reportes → "Proyección de Reabastecimiento" → "Generar proyección"
 - Resultado esperado: diasHastaNivelMinimo = 999 (sin datos de consumo reciente)

22. Manual técnico breve

- Se encuentra anexado en la carpeta de docs en archivo md.

23. Manual de usuario

- Se encuentra anexado en la carpeta de docs md.

24. Conclusiones

El desarrollo de Supply Manager nos permitió consolidar en la práctica los conceptos fundamentales de la ingeniería de software orientada a objetos. La implementación de una arquitectura de tres capas demostró ser una decisión acertada desde el inicio: al separar estrictamente la presentación, la lógica de negocio y el acceso a datos, fue posible modificar

los formularios sin tocar los servicios, y actualizar los procedimientos almacenados sin recompilar la aplicación, lo que evidencia el valor real de esta separación de responsabilidades.

La aplicación de los principios SOLID dejó de ser un concepto teórico para convertirse en una guía concreta de diseño. El principio de inversión de dependencias, en particular, resultó determinante: al hacer que los servicios dependan de la interfaz `IUnitOfWork` y no de una implementación concreta, el sistema quedó preparado para ser extendido o probado de forma aislada sin necesidad de modificar el código existente.

En cuanto a los patrones de diseño GoF, cada uno aportó una solución específica a un problema concreto. El Singleton garantizó un uso controlado de la conexión a base de datos; el Factory Method desacopló la creación de dependencias; el Repository hizo transparente el origen de los datos para las capas superiores; y el Unit of Work agrupó los repositorios en una única dependencia cohesiva para los servicios.

La decisión de centralizar las reglas de negocio críticas en procedimientos almacenados resultó especialmente valiosa. La validación de stock y la atomicidad del registro de movimientos en `sp\_RegistrarMovimiento` garantizan la integridad del inventario incluso si se accede a la base de datos desde otra herramienta, lo que demuestra que la robustez del sistema no depende exclusivamente del código de aplicación.

Finalmente, el reporte de proyección de reabastecimiento representó el aporte diferencial del proyecto: traducir una tasa de cambio discreta en una consulta SQL aplicable a un problema real de inventario es un ejemplo concreto de cómo los fundamentos matemáticos y de programación se complementan para generar valor en un sistema de gestión.

25. Mejoras futuras

1. Relación Insumo-Proveedor: implementar la tabla `InsumoProveedores` (N:M) para rastrear qué proveedor suministra cada insumo y generar órdenes de compra automáticas.
2. Exportación de reportes a Excel/PDF: integrar `ClosedXML` o `iTextSharp` para exportar los reportes generados a formato imprimible.
3. Histórico de precios: agregar la columna `PrecioUnitario` a `Insumos` y en los movimientos para calcular el valor monetario del inventario.
4. Cifrado de contraseñas: reemplazar el almacenamiento de contraseñas en texto plano por hashing con `SHA-256` o `BCrypt`.
5. Alertas en tiempo real: al iniciar sesión como Administrador o Almacenista, mostrar automáticamente los insumos con stock crítico en una notificación emergente.
6. Multi-instancia y pool de conexiones: migrar `DbConnectionSingleton` a `SqlConnectionStringBuilder` con pool de conexiones ADO.NET para soportar múltiples usuarios simultáneos (versión red local).
7. Módulo de órdenes de compra: formulario para generar y registrar órdenes de compra a proveedores, vinculadas a insumos en stock crítico.